

CS 305: Computer Networks

Fall 2025

Network Layer – The Control Plane

Ming Tang

Department of Computer Science and Engineering
Southern University of Science and Technology (SUSTech)

Chapter 5: outline

5.1 introduction

5.2 routing protocols

- ❖ link state (global)
- ❖ distance vector (decentralized)

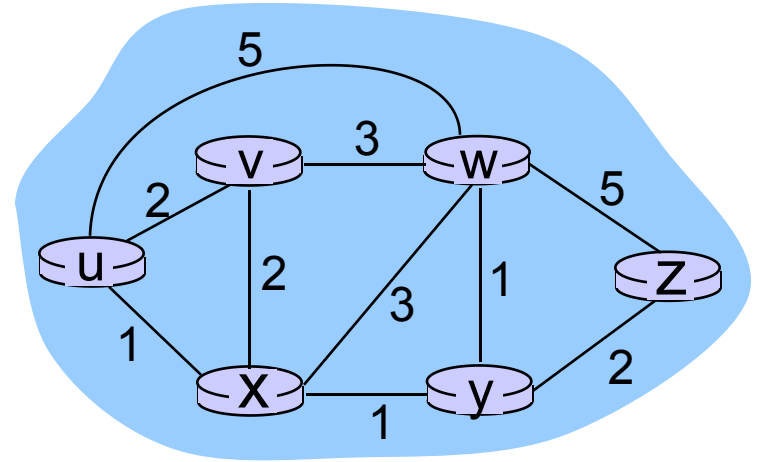
5.3 intra-AS routing in the Internet: OSPF

5.4 routing among the ISPs: BGP

5.5 The SDN control plane

5.6 ICMP: The Internet Control Message Protocol

5.7 Network management and SNMP



A link-state routing algorithm

Dijkstra's algorithm

- ❖ net topology, link costs known to all nodes
 - accomplished via “link state broadcast”
 - all nodes have same info
- ❖ computes least cost paths from one node (‘source’) to all other nodes
 - gives *forwarding table* for that node
- ❖ iterative: after k iterations, know least cost path to k dest.’s

notation:

- ❖ $c(x,y)$: link cost from node x to y; $= \infty$ if not direct neighbors
- ❖ $D(v)$: current value of cost of path from source to dest. v
- ❖ $p(v)$: predecessor node along path from source to v
- ❖ N' : set of nodes whose least cost path definitively known

Dijkstra's algorithm

1 **Initialization:**

2 $N' = \{u\}$

3 for all nodes v

4 if v adjacent to u

5 then $D(v) = c(u, v)$

6 else $D(v) = \infty$

7

8 **Loop**

9 find w not in N' such that $D(w)$ is a minimum

10 add w to N'

11 update $D(v)$ for all v adjacent to w and not in N' :

12 **$D(v) = \min(D(v), D(w) + c(w, v))$**

13 /* new cost to v is either old cost to v or known

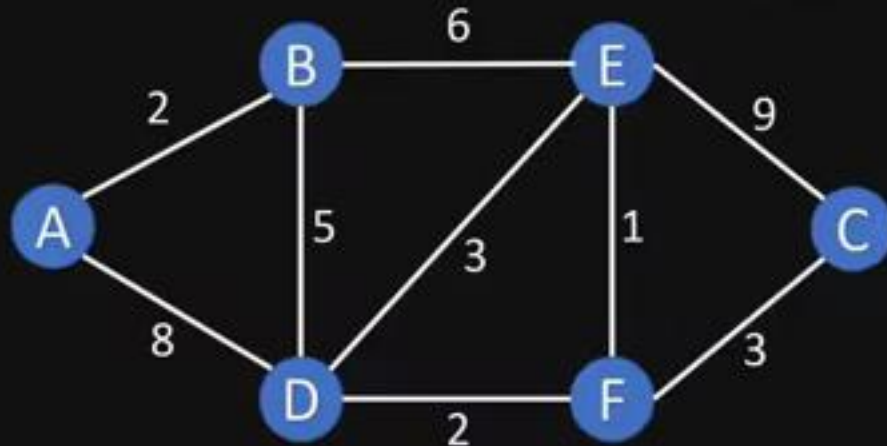
14 shortest path cost to w plus cost from w to v */

15 **until all nodes in N'**



Dijkstra's algorithm

Dijkstra's Shortest Path Algorithm



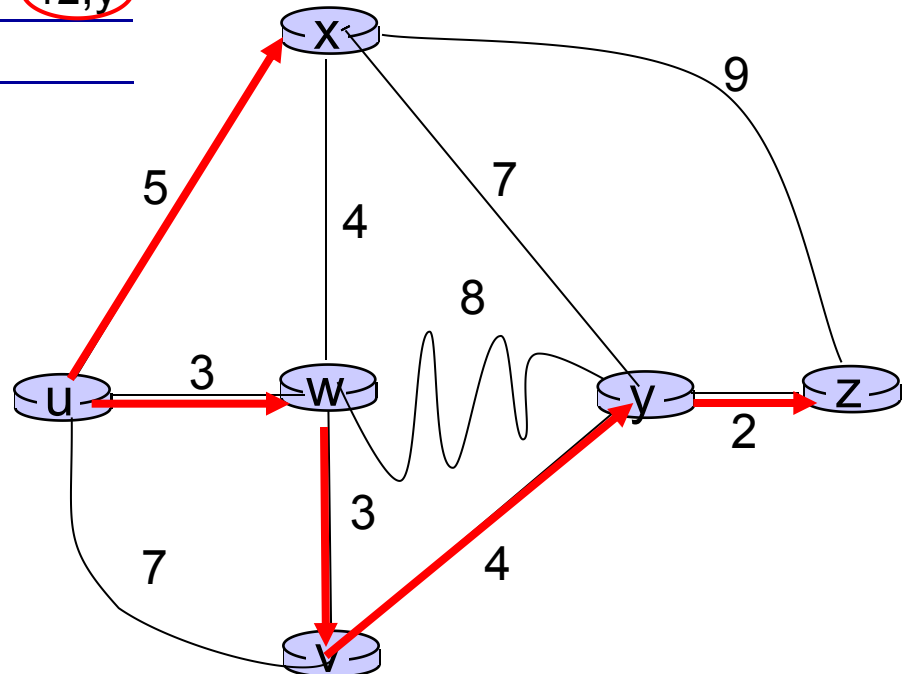
Dijkstra's Algorithm

Dijkstra's algorithm: example

Step	N'	D(v) p(v)	D(w) p(w)	D(x) p(x)	D(y) p(y)	D(z) p(z)
0	u	7,u	3,u	5,u	∞	∞
1	uw	6,w		5,u	11,w	∞
2	uwx	6,w			11,w	14,x
3	uwxv				10,v	14,x
4	uwxvy					12,y
5	uwxvyz					

notes:

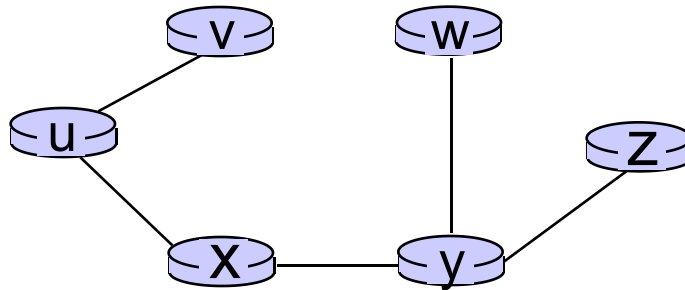
- ❖ construct shortest path tree by tracing predecessor nodes
- ❖ ties can exist (can be broken arbitrarily)



Dijkstra's algorithm: example

(2)

resulting shortest-path tree from u:



resulting forwarding table in u:

destination	link
v	(u,v)
x	(u,x)
y	(u,x)
w	(u,x)
z	(u,x)

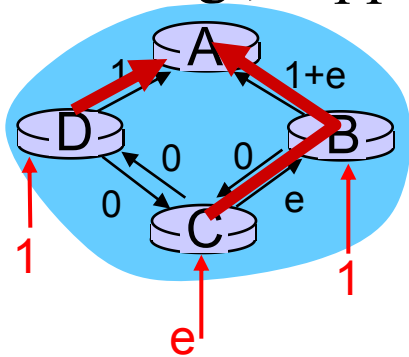
Dijkstra's algorithm, discussion

algorithm complexity: n nodes

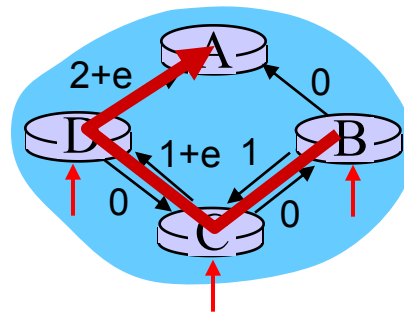
- ❖ each iteration: need to check all nodes not in N'
- ❖ $n(n+1)/2$ comparisons: $O(n^2)$
- ❖ more efficient implementations possible: $O(n \log n)$

Oscillations (摇摆) possible (if we consider congestion) :

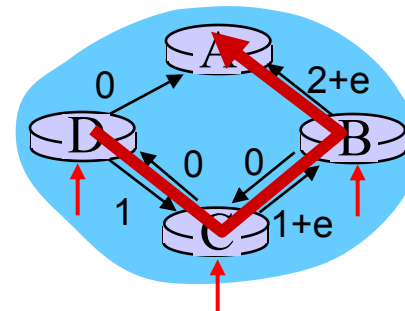
- ❖ e.g., support link cost equals amount of carried traffic:



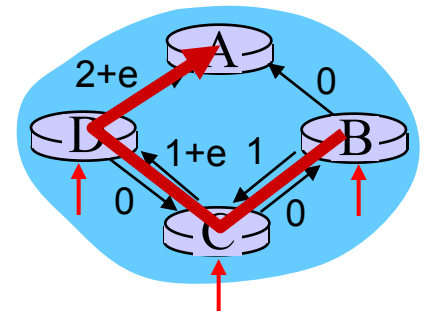
initially



given these costs,
find new routing....
resulting in new costs



given these costs,
find new routing....
resulting in new costs



given these costs,
find new routing....
resulting in new costs

Chapter 5: outline

5.1 introduction

5.2 routing protocols

- ❖ link state (global)
- ❖ distance vector (decentralized)

5.3 intra-AS routing in the Internet: OSPF

5.4 routing among the ISPs: BGP

5.5 The SDN control plane

5.6 ICMP: The Internet Control Message Protocol

5.7 Network management and SNMP

Distance vector algorithm

The distance-vector (DV) algorithm:

- **distributed**: each node **receives** some information from one or more of its directly attached neighbors, performs a **calculation**, and then **distributes** the results of its calculation back to its neighbors.
- **Iterative**: this process continues on until no more information is exchanged between neighbors.
- **Asynchronous**: it does not require all of the nodes to operate in lockstep with each other.

Bellman-Ford equation

Distance vector algorithm

Distance vector algorithm

Bellman-Ford equation:

$d_x(y) :=$ cost of least-cost path from x to y

then

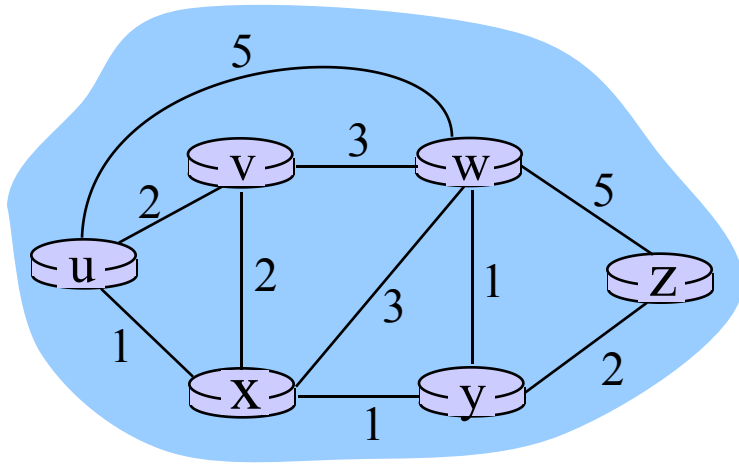
$$d_x(y) = \min_v \{ c(x,v) + d_v(y) \}$$

cost from neighbor v to destination y

cost to neighbor v

$\min$ taken over all neighbors v of x

Bellman-Ford example



clearly, $d_v(z) = 5$, $d_x(z) = 3$, $d_w(z) = 3$

B-F equation says:

$$\begin{aligned} d_u(z) &= \min \{ c(u,v) + d_v(z), \\ &\quad c(u,x) + d_x(z), \\ &\quad c(u,w) + d_w(z) \} \\ &= \min \{ 2 + 5, \\ &\quad 1 + 3, \\ &\quad 5 + 3 \} = 4 \end{aligned}$$

Node achieving minimum is

- the next hop in shortest path used in forwarding table

Distance vector algorithm

$D_x(y)$ = estimate of least cost from x to y

Node x:

- knows cost to each neighbor v: $c(x,v)$
- maintains its recent distance vector $\mathbf{D}_x = [D_x(y): y \in N]$
- maintains its neighbors' recent distance vectors. For each neighbor v, x maintains $\mathbf{D}_v = [D_v(y): y \in N]$

Distance vector algorithm

Key Idea:

- From time-to-time, each node sends its own recent distance vector (DV) to neighbors
- When x receives new DV from neighbor, it updates its own DV using B-F equation:

$$D_x(y) \leftarrow \min_v \{c(x,v) + D_v(y)\} \text{ for each node } y \in N$$

- If its DV has changed, sends the updated DV to neighbors
...

- ❖ under minor, natural conditions, the estimate $D_x(y)$ converge to the actual least cost $d_x(y)$

Distance vector algorithm

iterative, asynchronous:

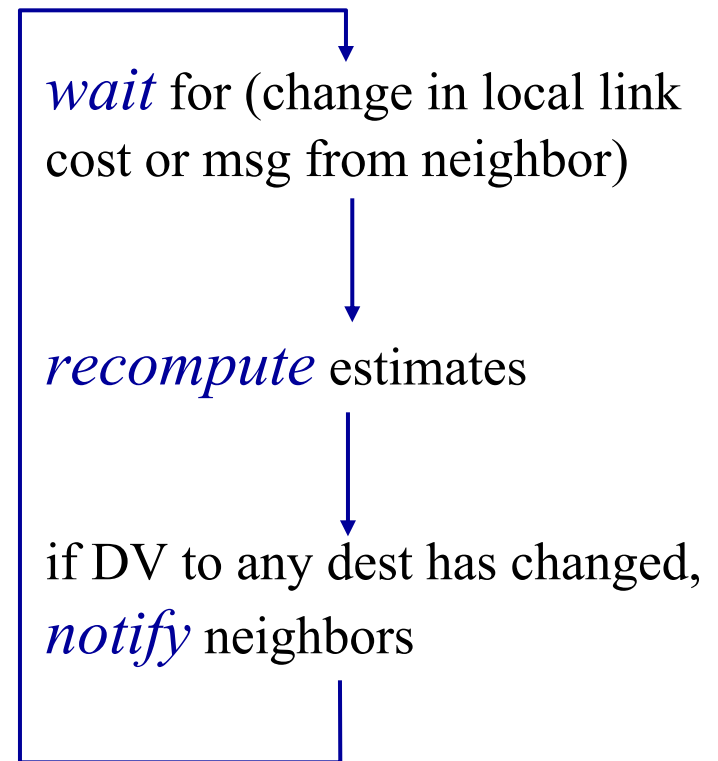
each local iteration caused by:

- local link cost change
- DV update message from neighbor

distributed:

- each node notifies neighbors *only* when its DV changes
 - neighbors then notify their neighbors if necessary

each node:



Distance vector algorithm

```
1  Initialization:
2    for all destinations  $y$  in  $N$ :
3       $D_x(y) = c(x, y)$  /* if  $y$  is not a neighbor then  $c(x, y) = \infty$  */
4    for each neighbor  $w$ 
5       $D_w(y) = ?$  for all destinations  $y$  in  $N$ 
6    for each neighbor  $w$ 
7      send distance vector  $\mathbf{D}_x = [D_x(y) : y \text{ in } N]$  to  $w$ 
8
9  loop
10    wait (until I see a link cost change to some neighbor  $w$  or
11           until I receive a distance vector from some neighbor  $w$ )
12
13    for each  $y$  in  $N$ :
14       $D_x(y) = \min_v \{c(x, v) + D_v(y)\}$ 
15
16    if  $D_x(y)$  changed for any destination  $y$ 
17      send distance vector  $\mathbf{D}_x = [D_x(y) : y \text{ in } N]$  to all neighbors
18
19  forever
```

$$D_x(y) = \min\{c(x,y) + D_y(y), c(x,z) + D_z(y)\}$$

$$= \min\{2+0, 7+1\} = 2$$

$$D_x(z) = \min\{c(x,y) + D_y(z), c(x,z) + D_z(z)\}$$

$$= \min\{2+1, 7+0\} = 3$$

**node x
table**

		cost to		
		x	y	z
from	x	0	2	7
	y	∞	∞	∞
	z	∞	∞	∞

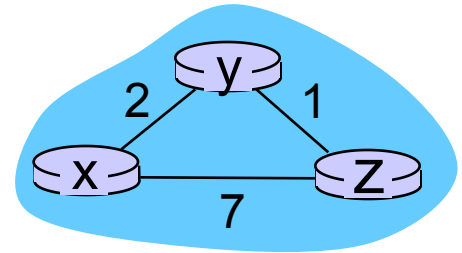
		cost to		
		x	y	z
from	x	0	2	3
	y	2	0	1
	z	7	1	0

**node y
table**

		cost to		
		x	y	z
from	x	∞	∞	∞
	y	2	0	1
	z	∞	∞	∞

**node z
table**

		cost to		
		x	y	z
from	x	∞	∞	∞
	y	∞	∞	∞
	z	7	1	0



time

$$D_x(y) = \min\{c(x,y) + D_y(y), c(x,z) + D_z(y)\}$$

$$= \min\{2+0, 7+1\} = 2$$

$$D_x(z) = \min\{c(x,y) + D_y(z), c(x,z) + D_z(z)\}$$

$$= \min\{2+1, 7+0\} = 3$$

**node x
table**

	cost to		
	x	y	z
from x	0	2	7
from y	∞	∞	∞
from z	∞	∞	∞

**node y
table**

	cost to		
	x	y	z
from x	∞	∞	∞
from y	2	0	1
from z	∞	∞	∞

**node z
table**

	cost to		
	x	y	z
from x	∞	∞	∞
from y	∞	∞	∞
from z	7	1	0

	cost to		
	x	y	z
from x	0	2	3
from y	2	0	1
from z	7	1	0

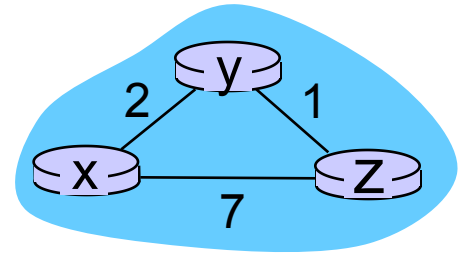
	cost to		
	x	y	z
from x	0	2	7
from y	2	0	1
from z	7	1	0

	cost to		
	x	y	z
from x	0	2	7
from y	2	0	1
from z	3	1	0

	cost to		
	x	y	z
from x	0	2	3
from y	2	0	1
from z	3	1	0

	cost to		
	x	y	z
from x	0	2	3
from y	2	0	1
from z	3	1	0

	cost to		
	x	y	z
from x	0	2	3
from y	2	0	1
from z	3	1	0

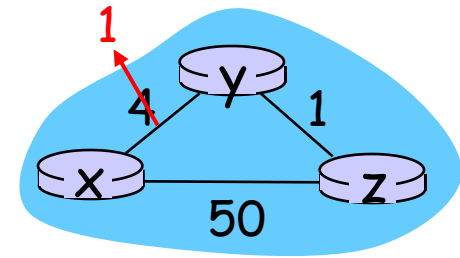


time →

Distance vector: link cost changes

link cost changes:

- ❖ node detects local link cost change
- ❖ updates routing info, recalculates distance vector
- ❖ if DV changes, notify neighbors



“good
news
travels
fast”

t_0 : y detects link-cost change, updates its DV, informs its neighbors.

t_1 : z receives update from y , updates its table, computes new least cost to x , sends its neighbors its DV.

t_2 : y receives z 's update, updates its distance table. y 's least costs do *not* change, so y does *not* send a message to z .

Distance vector: link cost changes

node x
table

		cost to		
		x	y	z
from	x	0	4 1	5 2
	y	4	0	1
	z	5	1	0

Detect $c(x,y)=c(y,x)=1$!

node y
table

		cost to		
		x	y	z
from	x	0	4	5
	y	4 1	0	1
	z	5	1	0

node z
table

		cost to		
		x	y	z
from	x	0	4	5
	y	4	0	1
	z	5	1	0

		cost to		
		x	y	z
from	x	0	1	2
	y	1	0	1
	z	5	1	0

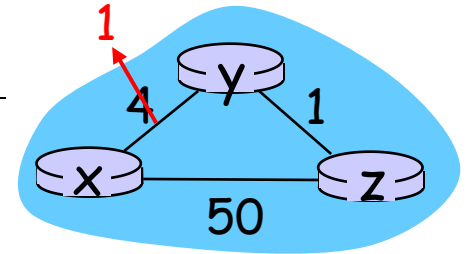
		cost to		
		x	y	z
from	x	0	1	2
	y	1	0	1
	z	5	1	0

		cost to		
		x	y	z
from	x	0	1	2
	y	1	0	1
	z	2	1	0

		cost to		
		x	y	z
from	x	0	1	2
	y	1	0	1
	z	2	1	0

		cost to		
		x	y	z
from	x	0	1	2
	y	1	0	1
	z	2	1	0

		cost to		
		x	y	z
from	x	0	1	2
	y	1	0	1
	z	2	1	0



$$D_x(z) = \min\{c(x,y) + D_y(z), c(x,z) + D_z(z)\}$$

“good news travels fast”

time

$$Dy(x) = \min\{c(y,x) + Dx(x), c(y,z) + Dz(x)\}$$

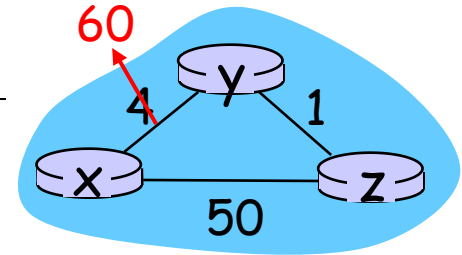
$$Dz(x) = \min\{c(z,x) + Dx(x), c(z,y) + Dy(x)\}$$

**node x
table**

		cost to		
		x	y	z
from	x	0	4	5
	y	4	0	1
	z	5	1	0

		cost to		
		x	y	z
from	x			
	y			
	z			

		cost to		
		x	y	z
from	x			
	y			
	z			



Detect $c(x,y) = c(y,x) = 60$!

**node y
table**

		cost to		
		x	y	z
from	x	0	4	5
	y	4	0	1
	z	5	1	0

		cost to		
		x	y	z
from	x	0	51	50
	y	6	0	1
	z	5	1	0

		cost to		
		x	y	z
from	x	0	51	50
	y	8	0	1
	z	7	1	0

**node z
table**

		cost to		
		x	y	z
from	x	0	4	5
	y	4	0	1
	z	5	1	0

		cost to		
		x	y	z
from	x	0	51	50
	y	6	0	1
	z	7	1	0

		cost to		
		x	y	z
from	x			
	y			
	z			

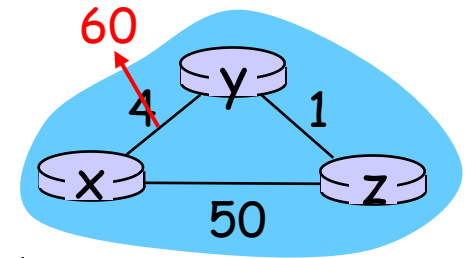
time

loop will persist for 44 iterations until z eventually computes the cost of its path via y to be greater than 50.

Distance vector: link cost changes

link cost changes:

- ❖ node detects local link cost change
- ❖ *bad news travels slow* - “count to infinity” problem!
- ❖ 44 iterations before algorithm stabilizes:
 - ❖ $D_y(x) = \min\{c(y,x) + D_x(x), c(y,z) + D_z(x)\} = \min\{60 + 0, 1 + 5\} = 6$
 - ❖ $D_z(x) = \min\{c(z,x) + D_x(x), c(z,y) + D_y(x)\} = \min\{50 + 0, 1 + 6\} = 7$
 - ❖ $D_y(x) = 8, D_z(x) = 9, \dots$ totally 44 iteration!



Poisoned reverse:

- ❖ If Z routes through Y to get to X :
 - Z tells Y its (Z's) distance to X is infinite (so Y won't route to X via Z)
- ❖ will this completely solve count to infinity problem?

$$Dy(x) = \min\{c(y,x) + Dx(x), c(y,z) + Dz(x)\}$$

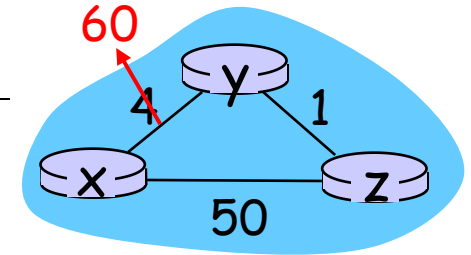
$$Dz(x) = \min\{c(z,x) + Dx(x), c(z,y) + Dy(x)\}$$

**node x
table**

		cost to		
		x	y	z
from	x	0	4	5
	y	4	0	1
	z	5	1	0

		cost to		
		x	y	z
from	x			
	y			
	z			

		cost to		
		x	y	z
from	x			
	y			
	z			



Detect $c(x,y) = c(y,x) = 60$!

**node y
table**

		cost to		
		x	y	z
from	x	0	4	∞
	y	4	0	1
	z	∞	1	0

		cost to		
		x	y	z
from	x	0	51	50
	y	60	0	1
	z	∞	1	0

		cost to		
		x	y	z
from	x	0	51	50
	y	51	0	1
	z	50	1	0

**node z
table**

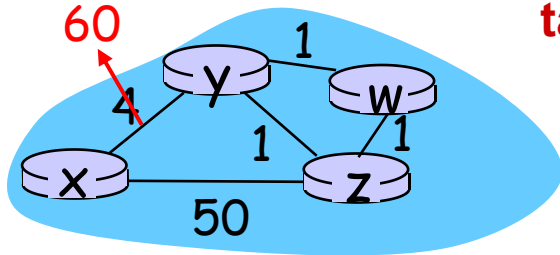
		cost to		
		x	y	z
from	x	0	4	5
	y	4	0	1
	z	5	1	0

		cost to		
		x	y	z
from	x	0	51	50
	y	60	0	1
	z	50	1	0

		cost to		
		x	y	z
from	x			
	y			
	z			

time

Distance vector: link cost changes



node x table

	cost to				
	x	y	z	w	
from	x	0	4	5	5
y	4	0	1	1	
z	5	1	0	1	

node y table

	cost to				
	x	y	z	w	
from	x	0	4	∞	∞
y	4	0	1	1	
z	∞	1	0	1	
w	∞	1	1	0	

node z table

	cost to				
	x	y	z	w	
from	x	0	4	5	5
y	4	0	1	1	
z	5	1	0	1	
w	5	1	1	0	

node z table

	cost to				
	x	y	z	w	
from	x	0	4	5	5
y	60	0	1	1	
z	6	1	0	1	
w	5	1	1	0	

node w table

	cost to				
	x	y	z	w	
from	y	60	0	1	1
z	5	1	0	1	
w	6	1	1	0	

node w table

	cost to				
	x	y	z	w	
from	y	4	0	1	1
z	5	1	0	1	
w	5	1	1	0	

node y table

	cost to				
	x	y	z	w	
from	x	0	4	∞	∞
y	7	0	1	1	
z	6	1	0	1	
w	6	1	1	0	

Poisoned reverse:

- ❖ will this completely solve count to infinity problem?
- ❖ No, when the loops involves three or more nodes

Comparison of LS and DV algorithms

message complexity

- **LS:** with n nodes, E links, $O(nE)$ msgs sent
- **DV:** exchange between neighbors only
 - convergence time varies

speed of convergence

- **LS:** $O(n^2)$ algorithm requires $O(nE)$ msgs
- **DV:** convergence time varies
 - may be routing loops
 - count-to-infinity problem

robustness: what happens if router malfunctions?

LS:

- node can advertise incorrect *link* cost
- each node computes only its *own* table

DV:

- DV node can advertise incorrect *path* cost
- each node's table used by others
 - error propagate thru network

Chapter 5: outline

5.1 introduction

5.2 routing protocols

- link state
- distance vector

5.3 intra-AS routing in the Internet: OSPF

5.4 routing among the ISPs: BGP

5.5 The SDN control plane

5.6 ICMP: The Internet Control Message Protocol

5.7 Network management and SNMP